Chapter 12: Linear Programming

Algorithms for Optimization, 2nd ed. (2026)

Kochenderfer & Wheeler

2026

Table of Contents

What is Linear Programming? I

Linear programming is one of the most widely used and practically successful branches of mathematical optimisation. A **linear program (LP)** is an optimisation problem in which *both* the objective function and every constraint are linear functions of the decision variables. Because everything is linear, the feasible region forms a geometric shape called a *convex polytope* — a multi-dimensional generalisation of a polygon — and the optimum is always attained at one of its vertices (corners).

Formal Definition

A linear program seeks to minimise a linear objective $c^\top x$ over a set of decision variables $x \in \mathbb{R}^n$, subject only to linear equality and inequality constraints:

$$\underset{x}{\text{minimize}}\ c^\top x \qquad \text{subject to linear equalities and/or inequalities in } x.$$

What is Linear Programming? II

Notation

- x (bold x) is the vector of n decision variables — the quantities we are choosing to optimise, e.g. amounts to produce or routes to take.
- c (bold c) is the *cost vector* of the same length n; its i-th entry c_i says how much the objective increases when x_i increases by one unit.
- $\mathbf{c}^\top \mathbf{x}$ (c transpose x) denotes the inner product $c_1x_1 + c_2x_2 + \cdots + c_nx_n$, a single scalar value.

Key Insight: Why Linear is Special

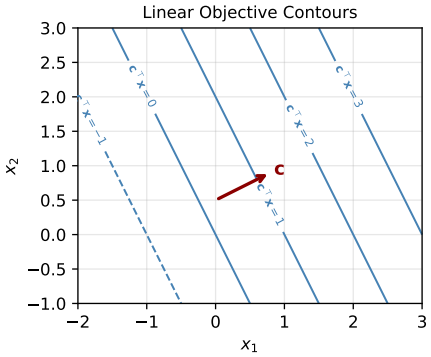
Because the objective surface is flat (a tilted plane), there are no local minima that are not also global minima. Any algorithm that reliably moves to a better point will eventually find the best point. This is the central reason LP problems can be solved efficiently even at very large scales.

What is Linear Programming? III

Applications of linear programming span an enormous range of practical fields. In *transportation and logistics*, LPs find minimum-cost shipping routes across a network. In *manufacturing and economics*, they allocate limited resources — labour hours, raw materials, machine time — to maximise profit or minimise waste. In *communication networks*, they route data flows to avoid congestion. In *statistics*, they arise when fitting models using the L_1 norm (sum of absolute residuals) or the L_∞ norm (minimax regression), both of which can be reformulated exactly as LPs, as we will see shortly.

Structural Properties

- The level sets (contours) of $c^\top x$ are parallel hyperplanes.
- The feasible set is a *convex polytope*: an intersection of half-spaces defined by the constraints.
- Any local minimum is automatically a global



Contours of $c^\top x$ are equally-spaced parallel lines. The cost vector c points in the direction of steepest increase; the optimum lies at the vertex where a contour line first touches the

12.1 General Form of a Linear Program I

Any LP can be written in the following *general form*, which allows a mixture of inequality and equality constraints.

General Form

$$\underset{x}{\text{minimize}}\ c^\top x \quad \text{subject to} \quad A_{\text{LE}}\, x \leq b_{\text{LE}}, \quad A_{\text{EQ}}\, x = b_{\text{EQ}}$$

12.1 General Form of a Linear Program II

Notation Explained

- $c \in \mathbb{R}^n$ and $x \in \mathbb{R}^n$ are the cost vector and the variable vector, both of length n (the number of decision variables).
- $A_{\mathrm{LE}} \in \mathbb{R}^{p \times n}$ is the matrix of p inequality-constraint coefficients; $b_{\mathrm{LE}} \in \mathbb{R}^p$ is the corresponding right-hand-side vector. The subscript "LE" stands for "less-than or equal to."
- $A_{\mathrm{EQ}} \in \mathbb{R}^{q \times n}$ is the matrix of q equality-constraint coefficients; $b_{\mathrm{EQ}} \in \mathbb{R}^q$ is the corresponding right-hand-side vector.
- The notation $Ax \leq b$ means that *every row* of Ax is at most the corresponding entry of b; that is, $(Ax)_i \leq b_i$ for all rows i.

How to Read This

Think of each inequality row $A_{\mathrm{LE},i}\, x \leq b_{\mathrm{LE},i}$ as a *budget constraint*: the weighted combination of the variables must not exceed the budget b_i. Every equality row $A_{\mathrm{EQ},i}\, x = b_{\mathrm{EQ},i}$ is a *balance equation*: the variables must sum (with given weights) to exactly b_i.

Example 12.1 — A Concrete LP

Consider the following LP with three variables x_1, x_2, x_3:

$$\underset{x_1, x_2, x_3}{\text{minimize}} \; 2x_1 - 3x_2 + 7x_3$$

subject to:

$$\begin{aligned} 2x_1 + 3x_2 - 8x_3 &\le 5 \\ 4x_1 + x_2 + 3x_3 &\le 9 \\ x_1 - 5x_2 - 3x_3 &\ge -4 \\ x_1 + x_2 + 2x_3 &= 1 \end{aligned}$$

The third constraint uses a "$\ge$" sign; to write it in standard form we multiply both sides by -1: $-x_1 + 5x_2 + 3x_3 \le 4$.

Example 12.2 — Norm Minimisation as an LP

The problem $\text{minimize}_{\mathrm{x}} \|\mathrm{Ax} - \mathrm{b}\|_1$ (minimise the sum of absolute residuals) is *not* in LP form directly because the absolute value function is not linear. However, we can introduce an auxiliary vector $\mathrm{s} \in \mathbb{R}^m$, $\mathrm{s} \ge 0$, that "bounds" each residual from above. The equivalent LP is:

$$\underset{\mathrm{x}, \mathrm{s}}{\text{minimize}} \; 1^\top \mathrm{s} \quad \text{s.t.} \quad \mathrm{Ax} - \mathrm{b} \le \mathrm{s}, \quad \mathrm{Ax} - \mathrm{b} \ge -\mathrm{s}$$

Here 1 is the all-ones vector, so $1^\top \mathrm{s}$ is the sum of all entries of s. Similarly, for the L_∞ norm $\text{minimize}_{\mathrm{x}} \|\mathrm{Ax} - \mathrm{b}\|_\infty$ (minimise the maximum absolute residual), we introduce a single scalar $t \ge 0$ and minimise it subject to:

$$\mathrm{Ax} - \mathrm{b} \le t1, \qquad \mathrm{Ax} - \mathrm{b} \ge -t1.$$

Standard Form and Converting Constraints I

Not every LP is written with only "$\leq$" constraints and non-negative variables. The *standard form* is a convenient intermediate representation that uses only "$\leq$" inequalities and enforces $x \geq 0$.

Standard Form

$$\underset{x}{\text{minimize}}\ c^\top x \quad \text{subject to} \quad Ax \leq b, \quad x \geq 0$$

All constraints are *less-than-or-equal* inequalities, and all variables are constrained to be *non-negative*.

Any LP in general form can be reduced to standard form using two elementary algebraic transformations.
Transformation 1: Equality to Two Inequalities. An equality constraint $A_{EQ}x = b_{EQ}$ can be replaced by a pair of inequalities that together enforce the same condition: the constraint holds if and only if *both* $A_{EQ}x \leq b_{EQ}$ and $-A_{EQ}x \leq -b_{EQ}$ hold simultaneously.

$$A_{EQ}x = b_{EQ} \longrightarrow A_{EQ}x \leq b_{EQ}, \qquad -A_{EQ}x \leq -b_{EQ}.$$

Transformation 2: Free Variables to Non-Negative Variables. If a variable x_i is *free* (can be positive or negative), we replace it with the difference of two non-negative variables: $x_i = x_i^+ - x_i^-$,

Standard Form and Converting Constraints II

where both $x_i^+ \geq 0$ and $x_i^- \geq 0$. Intuitively, x_i^+ captures the positive part of x_i and x_i^- captures the magnitude of the negative part. Doing this for every free variable x_j in x yields:

$$\underset{x^+,x^-}{\text{minimize}} \begin{bmatrix} c^\top & -c^\top \end{bmatrix} \begin{bmatrix} x^+ \\ x^- \end{bmatrix} \text{ s.t. } \begin{bmatrix} A & -A \end{bmatrix} \begin{bmatrix} x^+ \\ x^- \end{bmatrix} \leq b, \quad \begin{bmatrix} x^+ \\ x^- \end{bmatrix} \geq 0.$$

The cost of the negative part is $-c^\top x^-$ because increasing x_i^- by one unit *decreases* $x_i = x_i^+ - x_i^-$, which changes the objective by $-c_i$.

Key Insight

Every LP in any form can be converted to standard form, and standard form can then be converted to equality form (described next). This means the simplex algorithm, which requires equality form, is universally applicable.

Equality Form and Slack Variables I

The *equality form* is the representation required by the simplex algorithm. It has only equality constraints and non-negative variables.

Equality Form

$$\underset{x}{\text{minimize}}\ c^\top x \quad \text{subject to} \quad Ax = b, \quad x \geq 0$$

where $x \in \mathbb{R}^n$, $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$. There are m equality constraints and n non-negative variables, with $m \leq n$.

Equality Form and Slack Variables II

Converting Standard Form to Equality Form via Slack Variables

To convert the standard-form constraint $Ax \leq b$ to an equality, we introduce a *slack variable* vector $s \in \mathbb{R}^m$, $s \geq 0$. The slack $s_i \geq 0$ absorbs the gap between the left-hand side and the bound:

$$Ax \leq b \xrightarrow{\text{add } s \geq 0} Ax + s = b, \quad s \geq 0.$$

The combined equality-form LP over the augmented variable vector $[x^\top, s^\top]^\top$ is:

$$\underset{x,s}{\text{minimize}} \begin{bmatrix} c^\top & 0^\top \end{bmatrix} \begin{bmatrix} x \\ s \end{bmatrix} \text{ s.t. } \begin{bmatrix} A & I \end{bmatrix} \begin{bmatrix} x \\ s \end{bmatrix} = b, \quad \begin{bmatrix} x \\ s \end{bmatrix} \geq 0.$$

Here I is the $m \times m$ identity matrix and 0 is a zero vector of length m. The slacks appear in the objective with zero cost, so they do not change what we are optimising.

Equality Form and Slack Variables III

Notation

- s (bold s) is the *slack vector*; its i-th entry s_i measures how far constraint i is from being tight. When $s_i = 0$, constraint i is *active* (holding with equality); when $s_i > 0$, there is slack.
- I is the identity matrix; multiplying s by I simply writes each slack variable into its own row.

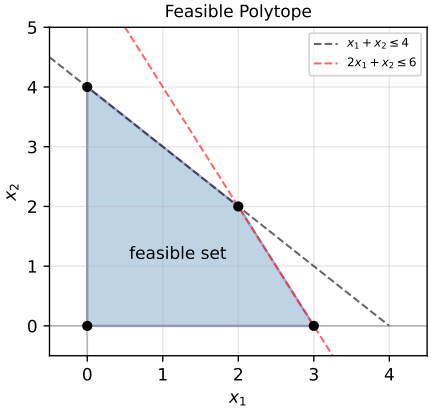
Equality Form and Slack Variables IV

Example 12.3 — Slack Variable in One Dimension

Consider the simple problem: $\text{minimize}_x\ x$ subject to $x \geq 1$. Multiplying the constraint by -1 gives $-x \leq -1$, so in standard form: $\text{minimize}_x\ x$ s.t. $-x \leq -1$, $x \geq 0$. Adding slack $s \geq 0$: $-x + s = -1$, which rearranges to $x - s = 1$. So the equality form is:

$$\underset{x,s}{\text{minimize}}\ x \quad \text{s.t.} \quad x - s = 1, \quad x, s \geq 0.$$

The constraint $x - s = 1$ restricts the feasible set to the line $x = 1 + s$ in the (x, s) plane. Since $x \geq 0$ and $s \geq 0$, the feasible set is the ray starting at $(x, s) = (1, 0)$ and extending upward. The minimum of x on this ray is at $x = 1$, $s = 0$.



The feasible polytope is the intersection of the half-spaces defined by all the constraints. Each constraint "cuts away" part of the space, and the remaining region is the set of valid solutions.

Geometric Interpretation of Linear Constraints I

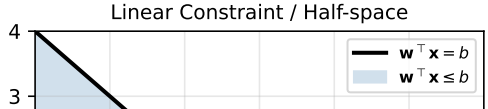
Understanding the geometry of LP is essential for understanding why the simplex algorithm works. Every linear constraint defines a *half-space*, and the feasible region is the intersection of finitely many such half-spaces.

Geometric Interpretation of Linear Constraints II

Half-Spaces and Hyperplanes

A single inequality constraint $\mathbf{w}^\top \mathbf{x} \leq b$ (where $\mathbf{w}$ is the constraint normal vector and b is a scalar bound) defines a *half-space*: the set of all points on one side of the hyperplane $\mathbf{w}^\top \mathbf{x} = b$.

- The hyperplane $\mathbf{w}^\top \mathbf{x} = b$ is perpendicular (orthogonal) to the normal vector $\mathbf{w}$.
- Points with $\mathbf{w}^\top \mathbf{x} < b$ lie on the *negative* side of the hyperplane — the side that satisfies the constraint.
- Points with $\mathbf{w}^\top \mathbf{x} > b$ lie on the *positive* side — the side that violates the constraint.



Four Possible Outcomes (Figure 12.4)

When solving an LP, exactly one of four outcomes must occur.

1. **Unique optimal solution.** The objective contour touches the feasible polytope at a single vertex. This is the most common case.
2. **Unbounded problem.** The feasible set extends to infinity in the direction of decreasing cost. This usually indicates a missing constraint.
3. **Infinitely many optimal solutions.** The objective contour is parallel to an entire face of the polytope, so all points on that face are equally optimal.

Geometric Interpretation of Linear Constraints III

Key Insight: Optima Live at Vertices

Because the objective $c^\top x$ is linear and the feasible set is a polytope, the minimum is always attained at a *vertex* of the polytope (unless the problem is unbounded or infeasible). This is why simplex — which visits vertices — is a complete method for LPs.

Example 12.4 — Converting to Equality Form Step by Step I

This example demonstrates how to convert a maximisation LP with free variables and inequality constraints into equality form, ready for the simplex algorithm.

Original Problem

Maximise $5x_1 + 4x_2$ subject to $2x_1 + 3x_2 \leq 5$, $4x_1 + x_2 \leq 11$, with x_1, x_2 free (can be any real number).

Example 12.4 — Converting to Equality Form Step by Step II

Step-by-Step Conversion

Step 1: Convert maximisation to minimisation. We want the minimum of the negated objective:

$$\underset{x_1, x_2}{\text{minimize}} \; -5x_1 - 4x_2.$$

Negating flips the sign of every coefficient: maximising f is the same as minimising $-f$.

Step 2: Introduce slack variables $s_1, s_2 \geq 0$. Add one slack variable per inequality constraint to convert "$\leq$" to "$=$":

$$2x_1 + 3x_2 + s_1 = 5,$$
$$4x_1 + \; x_2 + s_2 = 11.$$

The slack s_1 measures how far we are from using all of the first budget; s_2 measures the same for the second budget.

Step 3: Split free variables into positive and negative parts. Since x_1 and x_2 are free, we write $x_i = x_i^+ - x_i^-$ with $x_i^+, x_i^- \geq 0$. Substituting into the objective:

$$\text{minimize } -5(x_1^+ - x_1^-) - 4(x_2^+ - x_2^-) = -5x_1^+ + 5x_1^- - 4x_2^+ + 4x_2^-.$$

Substituting into the constraints:

Python: Setting Up and Solving an LP with SciPy

```
import numpy as np
from scipy.optimize import linprog

# -- Example 12.1 via scipy linprog ----------------------------------------
# minimize  2x1 - 3x2 + 7x3
# s.t.      2x1 + 3x2 - 8x3 <=  5
#           4x1 +  x2 + 3x3 <=  9
#          -x1 + 5x2 + 3x3  <=  4    (flipped from x1-5x2-3x3 >= -4)
#           x1 +  x2 + 2x3  ==  1    (equality constraint)

c      = np.array([ 2.0, -3.0,  7.0])
A_ub   = np.array([[ 2,  3, -8],
                   [ 4,  1,  3],
                   [-1,  5,  3]])
b_ub   = np.array([5.0, 9.0, 4.0])
A_eq   = np.array([[1, 1, 2]])
b_eq   = np.array([1.0])

res = linprog(c, A_ub=A_ub, b_ub=b_ub,
              A_eq=A_eq, b_eq=b_eq, method='highs')
print("Optimal x :", res.x)
print("Optimal f :", res.fun)

# -- L1 minimization as LP (Example 12.2) ----------------------------------
def l1_min_lp(A, b):
    """Minimize ||Ax - b||_1  via LP."""
    m, n = A.shape
```

12.2 The Simplex Algorithm — Core Idea I

The simplex algorithm, developed by George Dantzig in 1947, is the classic method for solving linear programs. Its core idea is geometrically elegant: *walk along the edges of the feasible polytope from vertex to vertex, always moving to a neighbouring vertex with a lower objective value, until no such improvement is possible.*

The Big Picture

Given an LP in equality form $Ax = b$, $x \geq 0$, the simplex algorithm proceeds in two phases:

1. **Initialisation phase.** Find any feasible vertex of the polytope to start from. This is done by solving a small auxiliary LP.
2. **Optimisation phase.** From the current vertex, check whether any neighbouring vertex has a lower objective. If yes, move there. If no, the current vertex is optimal — stop.

12.2 The Simplex Algorithm — Core Idea II

Requirements

The simplex algorithm requires:

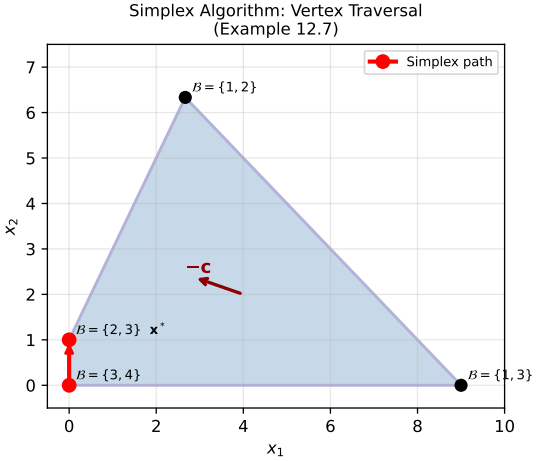
- The LP must be in **equality form**: $Ax = b$, $x \geq 0$.
- The rows of A must be **linearly independent**. If not, redundant rows should be removed.
- The number of constraints m must satisfy $m \leq n$ (we need at least as many variables as constraints).

12.2 The Simplex Algorithm — Core Idea III

Why does the optimum always lie at a vertex?
The objective $c^\top x$ is linear, so it has no curvature. Along any edge of the polytope, the objective either increases, decreases, or stays flat. If it decreases along an edge, we can walk to the other endpoint (the adjacent vertex) and have a better solution. We keep walking until no edge leads downhill — at that point we are at a vertex where all adjacent vertices are at least as expensive, which (for a convex polytope with a linear objective) means we have found the global minimum.

Convergence Guarantee

If the LP is feasible (has at least one valid point) and bounded (the objective does not go to $-\infty$), then the simplex algorithm is guaranteed to terminate at the global optimum. With an anti-cycling rule such as Bland's rule (see later), it terminates in a finite number of steps.



The simplex path on the feasible polytope for Example 12.7. The algorithm starts at the vertex corresponding to partition $\mathcal{B} = \{3, 4\}$ and moves to the optimal vertex $\mathcal{B} = \{2, 3\}$ in a single step.

12.2.1 Vertices and the Basis Partition I

Every vertex of the feasible polytope corresponds to a particular way of setting some variables to zero. Understanding this correspondence is the foundation of the simplex algorithm.

Vertex Characterisation

A *vertex* of the feasible polytope $\{x : Ax = b,\ x \geq 0\}$ is a point where exactly $n - m$ of the n variables are forced to zero. The remaining m variables may be positive (or zero, if the vertex is "degenerate"). We capture this by partitioning the index set $\{1, 2, \ldots, n\}$ into two disjoint sets:

$$\mathcal{B} \cup \mathcal{V} = \{1, 2, \ldots, n\}, \qquad |\mathcal{B}| = m, \quad |\mathcal{V}| = n - m.$$

- $\mathcal{B}$ (pronounced "cee-B", the *basic* or "busy" set): indices $i \in \mathcal{B}$ correspond to variables x_i that are *allowed* to be non-zero ("active"). There are exactly m basic indices, one per equality constraint.
- $\mathcal{V}$ (the *non-basic* or "vacant" set): indices $i \in \mathcal{V}$ correspond to variables forced to $x_i = 0$. There are $n - m$ such variables.

12.2.1 Vertices and the Basis Partition II

Extracting the Vertex Value

Because $x_i = 0$ for all $i \in \mathcal{V}$, the equality $\mathrm{A}\mathrm{x} = \mathrm{b}$ reduces to:

$$\mathrm{A}_{\mathcal{B}}\,\mathrm{x}_{\mathcal{B}} = \mathrm{b}, \qquad \Rightarrow \qquad \mathrm{x}_{\mathcal{B}} = \mathrm{A}_{\mathcal{B}}^{-1}\mathrm{b}.$$

Here $\mathrm{A}_{\mathcal{B}}$ is the $m \times m$ submatrix of A formed by selecting only the columns indexed by $\mathcal{B}$. For this to yield a unique vertex, $\mathrm{A}_{\mathcal{B}}$ must be *invertible* (non-singular).
The notation $\mathrm{A}_{\mathcal{B}}^{-1}$ means the matrix inverse of $\mathrm{A}_{\mathcal{B}}$. The solution $\mathrm{x}_{\mathcal{B}} = \mathrm{A}_{\mathcal{B}}^{-1}\mathrm{b}$ gives the values of the basic variables at this vertex; the non-basic variables are all zero.

How to Read This

This formula says: at a vertex, the m basic variables are determined by solving the m equality constraints simultaneously, treating the $n - m$ non-basic variables as zero. There are $\binom{n}{m}$ ways to choose $\mathcal{B}$, so there are at most $\binom{n}{m}$ vertices to examine — but the simplex algorithm navigates between them intelligently rather than checking all of them.

Example 12.6 — Verifying That a Point is a Vertex

Suppose we have the constraint matrix and candidate point:

$$A = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & -1 & 2 & 3 \\ 2 & 1 & 2 & -1 \end{bmatrix}, \quad x = [1, 1, 0, 0]^\top.$$

Here $n = 4$ variables and $m = 3$ constraints, so a vertex requires $n - m = 1$ variable to be zero. Indeed $x_3 = x_4 = 0$, so two variables are zero — this means the point is a vertex (possibly degenerate).
To confirm, try basis $\mathcal{B} = \{1, 2, 3\}$ (columns 1, 2, 3): $A_{\{1,2,3\}}$ is invertible. ✓
Also try $\mathcal{B} = \{1, 2, 4\}$ (columns 1, 2, 4): $A_{\{1,2,4\}}$ is invertible. ✓
Both bases reproduce $x_{\mathcal{B}} = [1, 1, 0]^\top$ and $[1, 1, 0]^\top$

Algorithm 12.1: Extract a Vertex

Given a partition $\mathcal{B}$:

1. Form the basis matrix $A_{\mathcal{B}}$ (columns of A at indices $\mathcal{B}$).
2. Check that $A_{\mathcal{B}}$ is invertible; if not, $\mathcal{B}$ does not define a valid vertex.
3. Solve $x_{\mathcal{B}} = A_{\mathcal{B}}^{-1} b$ for the basic variable values.
4. Set $x_i = 0$ for all $i \in \mathcal{V}$.
5. The resulting x is the vertex.

12.2.2 KKT Optimality Conditions at a Vertex I

To know whether the current vertex is optimal, we apply the *Karush–Kuhn–Tucker (KKT)* conditions. For linear programs, these first-order necessary conditions are also *sufficient* for global optimality.

The Lagrangian of the Equality-Form LP

We form the Lagrangian by attaching multipliers to the constraints. The Lagrangian $\mathcal{L}$ is:

$$\mathcal{L}(\mathrm{x}, \boldsymbol{\mu}, \boldsymbol{\lambda}) = \mathrm{c}^\top \mathrm{x} - \boldsymbol{\mu}^\top \mathrm{x} - \boldsymbol{\lambda}^\top (\mathrm{Ax} - \mathrm{b})$$

Notation

- $\boldsymbol{\mu}$ (bold mu, "mew") is a vector of n *non-negativity multipliers*, one per variable. Its i-th entry $\mu_i \geq 0$ penalises the constraint $x_i \geq 0$.
- $\boldsymbol{\lambda}$ (bold lambda, "lam-da") is a vector of m *Lagrange multipliers* for the equality constraints $\mathrm{Ax} = \mathrm{b}$. These are unrestricted in sign.
- The term $-\boldsymbol{\mu}^\top \mathrm{x}$ penalises variables that go below zero.
- The term $-\boldsymbol{\lambda}^\top (\mathrm{Ax} - \mathrm{b})$ penalises violations of the equality constraints.

12.2.2 KKT Optimality Conditions at a Vertex II

Four KKT Conditions (Necessary and Sufficient for LP)

At an optimal point, all four conditions must hold simultaneously:

1. **Primal feasibility**: $Ax = b$, $x \geq 0$ — the point is in the feasible set.
2. **Dual feasibility**: $\boldsymbol{\mu} \geq 0$ — non-negativity multipliers are non-negative.
3. **Complementary slackness**: $\boldsymbol{\mu} \odot x = 0$ — for every variable i, either $\mu_i = 0$ (the variable is active and free to vary) or $x_i = 0$ (the variable is at its lower bound). The symbol $\odot$ ("odot") denotes element-wise multiplication.
4. **Stationarity**: $A^\top \boldsymbol{\lambda} + \boldsymbol{\mu} = c$ — the gradient of the Lagrangian with respect to x is zero.

12.2.2 KKT Optimality Conditions at a Vertex III

Computing the Optimality Certificate at a Vertex. At a vertex defined by partition $(\mathcal{B}, \mathcal{V})$, the basic variables satisfy $x_{\mathcal{B}} \geq 0$ and the non-basic variables are $x_{\mathcal{V}} = 0$. Complementary slackness (Condition 3) therefore forces $\boldsymbol{\mu}_{\mathcal{B}} = 0$ (the basic variables are at positive values, so their multipliers must be zero). Substituting $\boldsymbol{\mu}_{\mathcal{B}} = 0$ into the stationarity condition $A^\top \boldsymbol{\lambda} + \boldsymbol{\mu} = c$ and looking at only the basic rows:

$$A_{\mathcal{B}}^\top \boldsymbol{\lambda} = c_{\mathcal{B}} \qquad \Rightarrow \qquad \boldsymbol{\lambda} = A_{\mathcal{B}}^{-\top} c_{\mathcal{B}}$$

where $A_{\mathcal{B}}^{-\top}$ means "the inverse of $A_{\mathcal{B}}$, then transposed." This gives us the Lagrange multipliers $\boldsymbol{\lambda}$ at the current vertex.

Next, applying the stationarity condition to the non-basic rows gives us the non-basic multipliers (also called *reduced costs*):

$$\boldsymbol{\mu}_{\mathcal{V}} = c_{\mathcal{V}} - \left(A_{\mathcal{B}}^{-1} A_{\mathcal{V}}\right)^\top c_{\mathcal{B}}.$$

12.2.2 KKT Optimality Conditions at a Vertex IV

Notation for the Reduced Cost Formula

- $c_{\mathcal{V}}$ is the sub-vector of cost coefficients at non-basic indices.
- $A_{\mathcal{V}}$ is the submatrix of A at non-basic columns.
- $A_{\mathcal{B}}^{-1} A_{\mathcal{V}}$ maps each non-basic column into basis coordinates.
- The reduced cost $(\boldsymbol{\mu}_{\mathcal{V}})_q$ tells us how much the objective changes when we increase non-basic variable x_q by one unit.

Optimality Test

- If $\boldsymbol{\mu}_{\mathcal{V}} \geq 0$ (all reduced costs are non-negative): the current vertex is **optimal**. Increasing any non-basic variable from zero can only increase the objective.
- If any $(\boldsymbol{\mu}_{\mathcal{V}})_q < 0$: the vertex is **suboptimal**. Increasing x_q from zero will decrease the objective. We should enter q into the basis — this is the *pivot* step.

12.2.3 The Pivot: Moving from Vertex to Vertex I

When the optimality test fails (some reduced cost is negative), we perform a *pivot*: we move along an edge of the polytope to an adjacent vertex with a lower objective value.

Choosing the Entering Variable (Index q)

We choose a non-basic index $q \in \mathcal{V}$ with $(\boldsymbol{\mu}_{\mathcal{V}})_q < 0$. Increasing x_q from zero will decrease the objective. Two common heuristics exist:

- **Dantzig's rule**: choose the index q with the *most negative* reduced cost. Intuition: this is the direction of steepest descent among all non-basic directions. Easy to compute but sensitive to variable scaling.
- **Bland's rule**: choose the *smallest index* q (by number) with a negative reduced cost. This is slower in practice but *guarantees termination* by preventing cycling (repeatedly visiting the same vertex without progress).

12.2.3 The Pivot: Moving from Vertex to Vertex II

Edge Transition: Computing Where to Go

Once we choose q, we increase x_q from 0 to some positive value x'_q. For the equality $Ax = b$ to remain satisfied, the basic variables must adjust. Specifically, define the *direction vector*:

$$d = A_{\mathcal{B}}^{-1} A_{\{q\}}$$

where $A_{\{q\}}$ is the q-th column of A. Then the new basic variable values are:

$$x'_{\mathcal{B}} = x_{\mathcal{B}} - d\, x'_q.$$

Notation

- d (the direction vector) has m entries. Its i-th entry d_i says how much the i-th basic variable decreases when x_q increases by one unit.
- We need $x'_{\mathcal{B}} \geq 0$ (all variables remain non-negative). As we increase x'_q, each basic variable with $d_i > 0$ decreases, eventually hitting zero.

The Minimum Ratio Test: Choosing the Leaving Variable (Index p)

To keep all basic variables non-negative, we must choose x'_q so that no basic variable goes below zero. The largest safe value of x'_q is:

$$x'_q = \min_{i\,:\,d_i>0} \frac{(x_{\mathcal{B}})_i}{d_i}$$

where the minimum is taken only over indices i with $d_i > 0$ (only those basic variables that decrease as x_q increases). The index $p \in \mathcal{B}$ achieving this minimum is the *leaving variable*: it is the first basic variable to hit zero as we walk along the edge.

12.2.3 The Pivot: Moving from Vertex to Vertex IV

Notation for the Minimum Ratio Test

- $(x_{\mathcal{B}})_i$ is the current value of the i-th basic variable.
- d_i is the i-th entry of the direction vector — how fast the i-th basic variable decreases per unit increase in x_q.
- The ratio $(x_{\mathcal{B}})_i / d_i$ is the "time" until variable i would reach zero if we keep increasing x_q. We take the minimum over all variables that are moving downward ($d_i > 0$).
- If all $d_i \leq 0$: no basic variable decreases as x_q increases, so we can increase x_q without bound — the LP is **unbounded**.

Updating the Partition

After determining p and x'_q, we swap p out of the basis and put q in:

$$\mathcal{B}' \leftarrow (\mathcal{B} \setminus \{p\}) \cup \{q\}.$$

The new partition $(\mathcal{B}', \mathcal{V}')$ defines the adjacent vertex we have moved to.

12.2.3 The Pivot: Moving from Vertex to Vertex V

Objective Change per Pivot

The change in objective from the pivot is:

$$c^\top x' - c^\top x = \mu_q \cdot x'_q$$

Since $\mu_q < 0$ (we chose q because its reduced cost is negative) and $x'_q > 0$ (we move a positive distance), the objective *decreases* at every non-degenerate pivot.

Cycling and Bland's Rule

A degenerate pivot occurs when $x'_q = 0$ (the leaving variable was already at zero), so the objective does not change. If such degenerate pivots repeat cyclically, the algorithm could run forever without progress. Bland's rule — always choosing the smallest eligible index — is proven to prevent cycling and guarantee finite termination.

Entering Index Heuristics — Summary

- **Greedy**: choose q to maximally reduce $c^\top x$ (requires computing all improvements).
- **Dantzig**: choose q with the most negative μ_q (easiest to compute, scaling-sensitive).
- **Bland**: choose the smallest q with $\mu_q < 0$ (prevents cycling, slower in practice).

Algorithm 12.3 and 12.4 (Python) — Simplex Step

```python
import numpy as np

def get_vertex(B, A, b):
    """Algorithm 12.1: Extract vertex from partition B."""
    b_inds = sorted(B)
    AB = A[:, b_inds]
    xB = np.linalg.solve(AB, b)
    x = np.zeros(A.shape[1])
    x[b_inds] = xB
    return x

def edge_transition(A, B, q):
    """Algorithm 12.2: Compute leaving index p and new x'_q."""
    n = A.shape[1]
    b_inds = sorted(B)
    n_inds = sorted(set(range(n)) - set(b_inds))
    AB = A[:, b_inds]
    xB = np.linalg.solve(AB, A[:, n_inds[q]])  # d = A_B^{-1} A_{q}
    return xB   # direction d

def step_lp(B, A, b, c):
    """Algorithm 12.3: One simplex step (greedy heuristic)."""
    n = A.shape[1]
    b_inds = sorted(B)
    n_inds = sorted(set(range(n)) - set(b_inds))
    AB, AV = A[:, b_inds], A[:, n_inds]
    xB   = np.linalg.solve(AB, b)
```

Example 12.7 — The Simplex Algorithm in Action I

We now trace through a complete execution of the simplex algorithm on a small LP to see exactly how it works.

Setup

The LP in equality form has the following data:

$$\mathrm{A} = \begin{bmatrix} 1 & 1 & 1 & 0 \\ -4 & 2 & 0 & 1 \end{bmatrix}, \quad \mathrm{b} = \begin{bmatrix} 9 \\ 2 \end{bmatrix}, \quad \mathrm{c} = \begin{bmatrix} 3 \\ -1 \\ 0 \\ 0 \end{bmatrix}.$$

We have $n = 4$ variables and $m = 2$ constraints, so the basis has $m = 2$ elements and there are $n - m = 2$ non-basic (zero) variables at each vertex. The initial partition uses the slack variables as the basis: $\mathcal{B} = \{3, 4\}$ (0-indexed: $\mathcal{B} = \{2, 3\}$ in 0-indexed Python).

Example 12.7 — The Simplex Algorithm in Action II

Iteration 1 — Starting at $\mathcal{B} = \{3, 4\}$

Step 1. Compute the basic variable values:

$$A_{\mathcal{B}} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad x_{\mathcal{B}} = A_{\mathcal{B}}^{-1} b = I\, b = \begin{bmatrix} 9 \\ 2 \end{bmatrix}.$$

So at this vertex: $x_1 = 0$, $x_2 = 0$ (non-basic, set to zero), $x_3 = 9$, $x_4 = 2$ (basic).

Step 2. Compute the Lagrange multipliers $\boldsymbol{\lambda}$ (lambda):

$$\boldsymbol{\lambda} = A_{\mathcal{B}}^{-\top} c_{\mathcal{B}} = I \begin{bmatrix} 0 \\ 0 \end{bmatrix} = 0.$$

Step 3. Compute the reduced costs $\boldsymbol{\mu}_{\mathcal{V}}$ for the non-basic variables $\mathcal{V} = \{1, 2\}$:

$$\boldsymbol{\mu}_{\mathcal{V}} = c_{\mathcal{V}} - A_{\mathcal{V}}^{\top} \boldsymbol{\lambda} = \begin{bmatrix} 3 \\ -1 \end{bmatrix} - 0 = \begin{bmatrix} 3 \\ -1 \end{bmatrix}.$$

Step 4. The reduced cost $\mu_2 = -1 < 0$, so the vertex is *not optimal*. We choose $q = 2$ (variable x_2) as the entering variable (it has negative reduced cost).

The Simplex Tableau I

The *simplex tableau* is a compact matrix representation of all the quantities needed for a simplex pivot. Rather than recomputing $A_{\mathcal{B}}^{-1}$ from scratch at each step, we maintain the tableau and update it incrementally via row operations.

Tableau Structure

At a vertex with partition $(\mathcal{B}, \mathcal{V})$, the simplex tableau contains three types of information:

	$x_{\mathcal{B}}$	$x_{\mathcal{V}}$	RHS
Constraint rows	I	$A_{\mathcal{B}}^{-1}A_{\mathcal{V}}$	$x_{\mathcal{B}} = A_{\mathcal{B}}^{-1}b$
Objective row	0	$\boldsymbol{\mu}_{\mathcal{V}}^{\top}$	$-c^{\top}x$ (negated objective)

- The **constraint rows** show the identity in the basic-variable columns (because we have "solved out" the basic variables) and the transformed non-basic columns $A_{\mathcal{B}}^{-1}A_{\mathcal{V}}$.
- The **right-hand side (RHS)** column gives the current basic variable values $x_{\mathcal{B}}$.
- The **objective row** (also called the "reduced cost row") contains the reduced costs $\boldsymbol{\mu}_{\mathcal{V}}$ in the non-basic columns. When all entries are non-negative, we are optimal.

The Simplex Tableau II

Pivoting in the Tableau

A pivot (swapping p out of $\mathcal{B}$ and q into $\mathcal{B}$) corresponds to a *column substitution* in the tableau: we perform standard row operations (Gaussian elimination) to make the q-column into a unit vector (with a 1 in the row of p and 0s elsewhere). This transforms the tableau into the representation at the new vertex without recomputing $\mathrm{A}_{\mathcal{B}}^{-1}$ from scratch.

Minimum Ratio Test in the Tableau

To introduce non-basic variable x_q into the basis, the direction of travel in $x_{\mathcal{B}}$-space is the column $d = A_{\mathcal{B}}^{-1} A_{\{q\}}$, which appears directly in the tableau. The minimum ratio test selects the leaving variable:

$$x'_q = \min_{i:\, d_i > 0} \frac{(x_{\mathcal{B}})_i}{d_i}.$$

- If all entries $d_i \leq 0$ in the pivot column: the LP is **unbounded** (we can increase x_q indefinitely without any basic variable going negative).
- When there are ties ($d_i/(x_{\mathcal{B}})_i$ equal for two or more i): the vertex is *degenerate* and the next step may not improve the objective. Bland's rule resolves ties by choosing the smallest index, preventing cycles.

12.2.4 Initialisation Phase — Finding a Feasible Vertex I

Before the simplex optimisation phase can run, we need a feasible starting vertex. Finding one is non-trivial: we cannot just set $x = 0$ unless $b = 0$, which is rarely the case. The standard approach is to solve an *auxiliary LP* whose feasible set is easy to start from and whose solution provides a feasible vertex for the original LP.

Auxiliary LP Construction

Introduce m *auxiliary variables* $z = [z_1, \ldots, z_m]^\top \geq 0$, one per equality constraint. The i-th auxiliary variable has coefficient Z_{ii}, where $Z_{ii} = +1$ if $b_i \geq 0$ and $Z_{ii} = -1$ if $b_i < 0$. The auxiliary LP is:

$$\underset{x,z}{\text{minimize}} \begin{bmatrix} 0^\top & 1^\top \end{bmatrix} \begin{bmatrix} x \\ z \end{bmatrix} \quad \text{s.t.} \quad \begin{bmatrix} A & Z \end{bmatrix} \begin{bmatrix} x \\ z \end{bmatrix} = b, \quad \begin{bmatrix} x \\ z \end{bmatrix} \geq 0$$

where $Z = \operatorname{diag}(Z_{11}, \ldots, Z_{mm})$ is the diagonal matrix of signs. The objective $1^\top z$ minimises the sum of all auxiliary variables, driving them toward zero.

12.2.4 Initialisation Phase — Finding a Feasible Vertex II

Notation

- 1 (ones vector) has all entries equal to 1, so $1^\top z = z_1 + z_2 + \cdots + z_m$.
- Z (diagonal matrix of signs) ensures that the initial point $x = 0$, $z_i = |b_i|$ is feasible for the auxiliary LP. This gives us an immediate starting vertex.
- The auxiliary LP has $n + m$ variables: the original n plus m auxiliary.

12.2.4 Initialisation Phase — Finding a Feasible Vertex III

Four-Step Procedure

1. Start with $\mathrm{x} = 0$ and $z_i = |b_i|$ for all i. The initial basis is $\mathcal{B} = \{n+1, n+2, \ldots, n+m\}$ (all auxiliary variables in the basis).
2. Run the simplex *optimisation phase* on the auxiliary LP.
3. If the optimal auxiliary objective is $1^\top \mathrm{z}^* = 0$: then $\mathrm{z}^* = 0$ and the current x^* is a feasible vertex for the original LP.
4. If $1^\top \mathrm{z}^* > 0$: it is *impossible* to satisfy all original constraints simultaneously. The original LP is **infeasible**.

After Step 3, any remaining auxiliary variables still in the basis (which must have value 0 at optimality) are swapped out for original x-variables to give a proper basis for the original LP.

Example 12.8 — Finding a Feasible Vertex

Consider the LP with two equality constraints: $2x_1 - x_2 + 2x_3 = 1$, $5x_1 + x_2 - 3x_3 = -2$. The right-hand sides are $b_1 = 1 \geq 0$ (so $Z_{11} = +1$) and $b_2 = -2 < 0$ (so $Z_{22} = -1$).
Auxiliary LP: add $z_1, z_2 \geq 0$: $2x_1 - x_2 + 2x_3 + z_1 = 1$, $5x_1 + x_2 - 3x_3 - z_2 = -2$.
Initial point: $\mathrm{x} = 0$, $z_1 = 1$, $z_2 = 2$. Initial basis: $\mathcal{B} = \{4, 5\}$ (the two auxiliary variables).
After running simplex on the auxiliary LP, we find: $\mathrm{x}^* \approx [0, 1, 1]^\top$ with $z_1 = z_2 = 0$. The new basis is $\mathcal{B} = \{2, 3\}$, and $[0, 1, 1]^\top$ is a feasible vertex of the original LP. ✓

Algorithm 12.5 (Python) — Complete Simplex Solver

```python
import numpy as np

def find_partition(A, b, c):
    """Algorithm 12.5: Find initial feasible vertex via auxiliary LP."""
    m, n = A.shape
    # Build Z: diagonal with +1 if b_i>=0 else -1
    Z = np.diag([1.0 if bi >= 0 else -1.0 for bi in b])
    A_aux = np.hstack([A, Z])
    b_aux = b.copy()
    c_aux = np.concatenate([np.zeros(n), np.ones(m)])
    # Initial partition: last m columns (z-variables), 0-indexed
    B = set(range(n, n + m))
    B, _ = minimize_lp(B, A_aux, b_aux, c_aux)
    # Replace z-indices in B with unused x-indices
    available_xs = sorted(set(range(n)) - B)
    zs_to_replace = sorted([j for j in B if j >= n])
    for k, j in enumerate(zs_to_replace):
        if k < len(available_xs):
            B = (B - {j}) | {available_xs[k]}
    return B

def minimize_lp(B, A, b, c):
    """Algorithm 12.4: Minimize LP from initial partition B."""
    done = False
    while not done:
        B, done = step_lp(B, A, b, c)
    x = get_vertex(B, A, b)
```

Python: Practical LP Solving with SciPy

```python
from scipy.optimize import linprog
import numpy as np

# -- Example 12.7 via scipy (standard form) ---------------------------------
# LP: min  3x1 - x2   s.t. x1+x2<=9, -4x1+2x2<=2, x1,x2>=0
c      = np.array([3.0, -1.0])
A_ub   = np.array([[ 1.0,  1.0],
                   [-4.0,  2.0]])
b_ub   = np.array([9.0, 2.0])
bounds = [(0, None), (0, None)]     # x1, x2 >= 0

result = linprog(c, A_ub=A_ub, b_ub=b_ub,
                 bounds=bounds, method='highs')
print("Optimal x  :", result.x)
print("Optimal obj:", round(result.fun, 4))
# Optimal x  : [0. 1.]
# Optimal obj: -1.0

# -- Dual variables (Lagrange multipliers) ----------------------------------
# HiGHS method returns dual values via result.ineqlin.marginals
print("Dual (lambda):", result.ineqlin.marginals)

# -- Example: L-infinity minimization ---------------------------------------
def linf_regression(A, b):
    """Solve min_x ||Ax - b||_inf  as an LP."""
    m, n = A.shape
    # Variables: [x (n), t (1)].  Minimize t.
```

Simplex Complexity and Practical Considerations I

The simplex algorithm has been the workhorse of LP solving for over 75 years. Understanding its theoretical properties and practical behaviour helps us use it wisely.

Computational Cost per Iteration

Each simplex step requires:

1. Solving $\mathrm{A}_{\mathcal{B}}^{\top}\boldsymbol{\lambda} = \mathrm{c}_{\mathcal{B}}$ for $\boldsymbol{\lambda}$ (lambda): costs $O(m^2)$ with LU factorisation.
2. Computing the reduced costs $\boldsymbol{\mu}_{\mathcal{V}} = \mathrm{c}_{\mathcal{V}} - \mathrm{A}_{\mathcal{V}}^{\top}\boldsymbol{\lambda}$: costs $O(mn)$.
3. Computing the direction $\mathrm{d} = \mathrm{A}_{\mathcal{B}}^{-1}\mathrm{A}_{\{q\}}$: costs $O(m^2)$ using the stored LU factors.
4. The minimum ratio test: costs $O(m)$.

Total per iteration: $O(m^2 n)$ in the dense case. In practice, A is often *sparse* (most entries are zero), which reduces the cost substantially.

Simplex Complexity and Practical Considerations II

Worst-Case vs. Average-Case Behaviour

The simplex algorithm has a *worst-case exponential* number of steps: Klee and Minty (1972) constructed a family of LPs with n variables where simplex visits all 2^n vertices before finding the optimum. However, in the vast majority of practical problems, simplex terminates in $O(m)$ to $O(mn)$ steps, which is polynomial. This average-case polynomial behaviour is why simplex remains the method of choice for many applications.

Interior-Point Methods: An Alternative

Interior-point methods (also called *barrier methods*) approach the optimum from the *interior* of the feasible polytope rather than walking along its edges. Their worst-case complexity is polynomial: $O(n^{3.5})$ for the classical algorithms. For very large LPs (millions of variables), interior-point methods can outperform simplex.

Simplex Variants

- **Revised simplex**: maintains and updates only the basis inverse $A_{\mathcal{B}}^{-1}$ incrementally, using LU factorisation with periodic refactorisation for numerical stability.
- **Dual simplex**: operates on the dual LP; useful for warm-starting after small changes to the problem.
- **Self-dual simplex**: avoids the two-phase structure by combining primal and dual

Step Summary: One Complete Simplex Iteration

1. Compute basic variable values: $\mathrm{x}_{\mathcal{B}} = \mathrm{A}_{\mathcal{B}}^{-1}\mathrm{b}$.
2. Compute Lagrange multipliers: $\boldsymbol{\lambda} = \mathrm{A}_{\mathcal{B}}^{-\top}\mathrm{c}_{\mathcal{B}}$.
3. Compute reduced costs: $\boldsymbol{\mu}_{\mathcal{V}} = \mathrm{c}_{\mathcal{V}} - \mathrm{A}_{\mathcal{V}}^{\top}\boldsymbol{\lambda}$.
4. **If** $\boldsymbol{\mu}_{\mathcal{V}} \geq 0$: **STOP** — current vertex is optimal.
5. Choose entering index q: pick any $(\boldsymbol{\mu}_{\mathcal{V}})_q < 0$ (e.g., Dantzig's rule: the most negative; Bland's rule: the smallest index).
6. Compute direction: $\mathrm{d} = \mathrm{A}_{\mathcal{B}}^{-1}\mathrm{A}_{\{q\}}$.
7. Choose leaving index p via minimum ratio test: $p = \arg\min_{i:d_i>0} (\mathrm{x}_{\mathcal{B}})_i / d_i$.
8. Swap: $\mathcal{B} \leftarrow (\mathcal{B} \setminus \{p\}) \cup \{q\}$.

12.3 Dual Linear Programs and Strong Duality I

Every linear program has a companion problem called the *dual LP*. The dual provides an independent lower bound on the primal objective: for any dual-feasible $\boldsymbol{\lambda}$ (lambda, pronounced "lam-da"), the dual objective $b^\top \boldsymbol{\lambda}$ is a lower bound on the optimal primal objective $c^\top x^*$. Remarkably, for linear programs, this lower bound is tight: at optimality, primal and dual objectives are exactly equal.

The Primal–Dual Pair

The primal LP minimises a linear objective over the feasible set; the dual LP maximises a related objective subject to dual constraints. Together they form a complementary pair:

- **Primal (P):** $\underset{x}{\text{minimize}}\ c^\top x$ subject to $Ax = b$, $x \geq 0$.
- **Dual (D):** $\underset{\boldsymbol{\lambda}}{\text{maximize}}\ b^\top \boldsymbol{\lambda}$ subject to $A^\top \boldsymbol{\lambda} \leq c$, $\boldsymbol{\lambda}$ free (unrestricted in sign).
- **Strong duality:** For any LP in equality form, the primal and dual share the same optimal objective value: $p^* = d^*$.

12.3 Dual Linear Programs and Strong Duality II

Notation for the Dual

- $\boldsymbol{\lambda}$ (bold lambda) is the vector of m dual variables, one per equality constraint of the primal. In economics, λ_i is the *shadow price* of constraint i: it measures how much the optimal objective changes if we relax constraint i by one unit.
- $\mathrm{A}^\top \boldsymbol{\lambda} \leq \mathrm{c}$ means: for each $j = 1, \ldots, n$, the j-th column of A dotted with $\boldsymbol{\lambda}$ must not exceed c_j. These are the dual feasibility constraints.
- The dual has m variables and n inequality constraints — exactly the reverse of the primal (which has n variables and m equality constraints).

How to Read Strong Duality

Strong duality says the minimum cost of the primal problem equals the maximum lower bound provided by any dual solution. In other words: there is no duality gap — no room between the best lower bound and the actual optimal value. This means we can verify a primal solution is optimal by finding a dual solution that achieves the same objective value, without re-solving anything.

Deriving the Dual via the Lagrangian

Starting from the equality-form primal, the Lagrangian $\mathcal{L}$ is formed by attaching multipliers $\boldsymbol{\mu}$ (mu, "mew") to the non-negativity constraints and $\boldsymbol{\lambda}$ (lambda) to the equality constraints:

$$\mathcal{L}(\mathrm{x}, \boldsymbol{\mu}, \boldsymbol{\lambda}) = \mathrm{c}^\top \mathrm{x} - \boldsymbol{\mu}^\top \mathrm{x} - \boldsymbol{\lambda}^\top (\mathrm{A}\mathrm{x} - \mathrm{b}) = (\mathrm{c} - \boldsymbol{\mu} - \mathrm{A}^\top \boldsymbol{\lambda})^\top \mathrm{x} + \boldsymbol{\lambda}^\top \mathrm{b}.$$

The *dual function* is the infimum of the Lagrangian over x. Because the Lagrangian is linear in x, its infimum is $\boldsymbol{\lambda}^\top \mathrm{b}$ when the coefficient $(\mathrm{c} - \boldsymbol{\mu} - \mathrm{A}^\top \boldsymbol{\lambda})$ equals zero; otherwise the infimum is $-\infty$ (we can make x arbitrarily large in the right direction). For the dual function to be finite, we therefore need $\mathrm{c} - \boldsymbol{\mu} - \mathrm{A}^\top \boldsymbol{\lambda} = 0$. Combined with $\boldsymbol{\mu} \geq 0$ (mu is non-negative), this gives $\mathrm{A}^\top \boldsymbol{\lambda} = \mathrm{c} - \boldsymbol{\mu} \leq \mathrm{c}$. Maximising the dual function $\mathrm{b}^\top \boldsymbol{\lambda}$ subject to $\mathrm{A}^\top \boldsymbol{\lambda} \leq \mathrm{c}$ is precisely the dual problem.

Weak and Strong Duality

Weak duality (always holds for any primal-feasible x and dual-feasible $\boldsymbol{\lambda}$):

$$\mathrm{b}^\top \boldsymbol{\lambda} \leq \mathrm{c}^\top \mathrm{x}.$$

The dual objective is always a lower bound on the primal objective, at any feasible pair — not just at the optimum.

Strong duality (LP-specific, at optimality): $p^* = d^*$. The Lagrange multipliers $\boldsymbol{\lambda}^*$ computed by the simplex algorithm (via $\boldsymbol{\lambda} = \mathrm{A}_{\mathcal{B}}^{-\top} \mathrm{c}_{\mathcal{B}}$) are automatically the optimal dual variables, providing a dual certificate "for free" at the end of the simplex run.

Dual Certificates: Verifying Optimality I

A *dual certificate* is a pair $(x^*, \boldsymbol{\lambda}^*)$ that proves the optimality of x^* without re-solving the LP from scratch. This is extremely useful for *verification*: a third party can check the certificate cheaply even if they did not solve the LP themselves.

Algorithm 12.6: Three-Condition Optimality Check

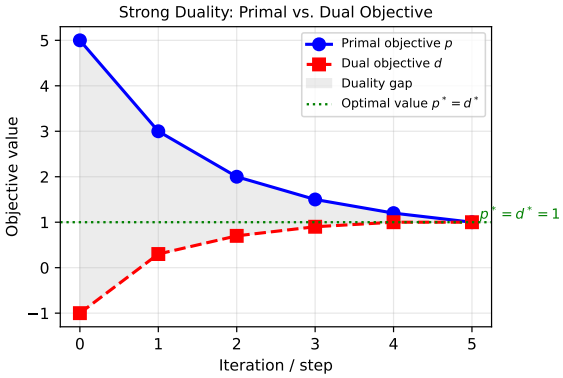
A pair $(x^*, \boldsymbol{\lambda}^*)$ is an optimal primal–dual pair if and only if all three conditions hold simultaneously:

1. **Primal feasibility**: $Ax^* = b$ and $x^* \geq 0$. The candidate solution x^* lies in the feasible set.
2. **Dual feasibility**: $A^\top \boldsymbol{\lambda}^* \leq c$. The dual candidate $\boldsymbol{\lambda}^*$ satisfies the dual constraints.
3. **Zero duality gap**: $c^\top x^* = b^\top \boldsymbol{\lambda}^*$. The primal and dual objectives match at the candidate pair.

Key Insight: Why Three Conditions Suffice

By weak duality, we always have $b^\top \boldsymbol{\lambda}^* \leq c^\top x^*$ for any dual-feasible $\boldsymbol{\lambda}^*$ and primal-feasible x^*. If additionally the objectives are equal (condition 3), then there can be no better primal solution: any feasible x satisfies $c^\top x \geq b^\top \boldsymbol{\lambda}^* = c^\top x^*$, so x^* is already at the minimum.

Dual Certificates: Verifying Optimality II



As the simplex algorithm iterates, the primal objective decreases and the dual objective increases, converging to the common optimal value $p^* = d^*$. Strong duality guarantees they meet exactly.

Practical Significance

Dual certificates are important in several contexts:

- **Verification**: check a claimed solution without re-solving.
- **Sensitivity analysis**: the dual variables $\boldsymbol{\lambda}^*$ are *shadow prices* that tell us how the optimal cost changes when each constraint is relaxed by a small amount.
- **Warm starting**: if the problem data changes slightly, the dual solution from the previous run often provides a good starting point for the next run.
- **Infeasibility detection**: if the dual problem is unbounded, the primal is infeasible.

Example 12.9 — Verifying a Solution with a Dual Certificate I

We now work through the three-condition check on a concrete numerical example, computing each step explicitly.

Given LP (Equality Form)

$$A = \begin{bmatrix} 1 & 1 & -1 \\ -1 & 2 & 0 \\ 1 & 2 & 3 \end{bmatrix}, \quad b = \begin{bmatrix} 1 \\ -2 \\ 5 \end{bmatrix}, \quad c = \begin{bmatrix} 1 \\ 1 \\ -1 \end{bmatrix}.$$

Candidate primal solution: $x^* = [2, 0, 1]^\top$. Candidate dual multipliers: $\boldsymbol{\lambda}^* = [1, 0, 0]^\top$.

Condition 1: Primal Feasibility. Check $Ax^* = b$ and $x^* \geq 0$.
Computing Ax^*:

$$Ax^* = \begin{bmatrix} 1 \cdot 2 + 1 \cdot 0 + (-1) \cdot 1 \\ (-1) \cdot 2 + 2 \cdot 0 + 0 \cdot 1 \\ 1 \cdot 2 + 2 \cdot 0 + 3 \cdot 1 \end{bmatrix} = \begin{bmatrix} 2 + 0 - 1 \\ -2 + 0 + 0 \\ 2 + 0 + 3 \end{bmatrix} = \begin{bmatrix} 1 \\ -2 \\ 5 \end{bmatrix} = b. \checkmark$$

Example 12.9 — Verifying a Solution with a Dual Certificate II

Also, $x^* = [2, 0, 1]^\top \geq 0$. ✓ Primal feasibility is confirmed.
Condition 2: Dual Feasibility. Check $A^\top \boldsymbol{\lambda}^* \leq c$.
Computing $A^\top \boldsymbol{\lambda}^*$:

$$A^\top \boldsymbol{\lambda}^* = \begin{bmatrix} 1 & -1 & 1 \\ 1 & 2 & 2 \\ -1 & 0 & 3 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ -1 \end{bmatrix}.$$

We need $A^\top \boldsymbol{\lambda}^* \leq c = [1, 1, -1]^\top$. Checking entry by entry: $1 \leq 1$ ✓, $1 \leq 1$ ✓, $-1 \leq -1$ ✓. Dual feasibility is confirmed.
Condition 3: Zero Duality Gap.

$$p^* = c^\top x^* = 1 \cdot 2 + 1 \cdot 0 + (-1) \cdot 1 = 2 + 0 - 1 = 1.$$

$$d^* = b^\top \boldsymbol{\lambda}^* = 1 \cdot 1 + (-2) \cdot 0 + 5 \cdot 0 = 1.$$

$p^* = d^* = 1$. ✓ Equal objectives are confirmed.

Example 12.9 — Verifying a Solution with a Dual Certificate III

Conclusion

All three conditions hold, so $x^* = [2, 0, 1]^\top$ is the optimal solution of this LP, and $\boldsymbol{\lambda}^* = [1, 0, 0]^\top$ is the optimal dual solution. The optimal value is 1.

Python: Dual Certificate Verification

```python
import numpy as np

def dual_certificate(A, b, c, x_star, lam_star, atol=1e-6):
    """
    Algorithm 12.6 (Python): Verify (x*, lam*) is an optimal
    solution pair for the equality-form LP:
        min  c^T x  s.t.  A x = b,  x >= 0.
    Returns True iff all three conditions hold.
    """
    # 1. Primal feasibility
    primal_feasible = (
        np.allclose(A @ x_star, b, atol=atol) and
        np.all(x_star >= -atol)
    )
    # 2. Dual feasibility
    dual_feasible = np.all(A.T @ lam_star <= c + atol)

    # 3. Equal objectives (zero duality gap)
    p_star = c    @ x_star
    d_star = b    @ lam_star
    equal  = np.isclose(p_star, d_star, atol=atol)

    print("Primal feasible :", primal_feasible)
    print("Dual feasible   :", dual_feasible)
    print("p* =", round(p_star, 6), " d* =", round(d_star, 6), " equal =", equal)
    return primal_feasible and dual_feasible and equal
```

12.4 Chapter Summary I

This chapter has covered the theory and algorithms for solving linear programs, from problem formulation through the simplex algorithm to duality theory.

Key Concept 1: Problem Formulation

A linear program (LP) minimises a linear objective $c^\top x$ subject to linear constraints. Any LP can be converted to *equality form* $Ax = b$, $x \geq 0$ using slack variables and variable splitting. The feasible set is a convex polytope, and the global minimum is always attained at one of its vertices.

Key Concept 2: The Simplex Algorithm

The simplex algorithm traverses vertices of the feasible polytope, performing *pivots* to move to adjacent vertices with lower objective values. Each vertex is represented by a basis partition $(\mathcal{B}, \mathcal{V})$; optimality is confirmed when all reduced costs $\boldsymbol{\mu}_{\mathcal{V}} \geq 0$. The algorithm runs in two phases: an *initialisation phase* (finds a feasible vertex using an auxiliary LP) and an *optimisation phase* (pivots until optimal). With Bland's rule, convergence is guaranteed in a finite number of steps.

12.4 Chapter Summary II

Key Concept 3: Duality and Certificates

Every LP has a dual LP. *Strong duality* guarantees $p^* = d^*$ — the primal minimum equals the dual maximum with no duality gap. A pair $(x^*, \boldsymbol{\lambda}^*)$ is optimal if and only if it is primal feasible, dual feasible, and the objectives match. The simplex algorithm automatically computes the optimal dual variables $\boldsymbol{\lambda}^* = A_{\mathcal{B}}^{-\top} c_{\mathcal{B}}$.

12.4 Chapter Summary III

Simplex Algorithm — Quick Reference

At each vertex $(\mathcal{B}, \mathcal{V})$:

1. $x_{\mathcal{B}} = A_{\mathcal{B}}^{-1} b$ (basic values)
2. $\boldsymbol{\lambda} = A_{\mathcal{B}}^{-\top} c_{\mathcal{B}}$ (Lagrange multipliers)
3. $\boldsymbol{\mu}_{\mathcal{V}} = c_{\mathcal{V}} - A_{\mathcal{V}}^{\top} \boldsymbol{\lambda}$ (reduced costs)
4. If $\boldsymbol{\mu}_{\mathcal{V}} \geq 0$: **optimal**
5. Else choose q with $\mu_q < 0$ (entering variable)
6. $d = A_{\mathcal{B}}^{-1} A_{\{q\}}$ (direction)
7. $p = \arg\min_{d_i > 0} (x_{\mathcal{B}})_i / d_i$ (leaving variable)
8. $\mathcal{B} \leftarrow (\mathcal{B} \setminus \{p\}) \cup \{q\}$

Practical Tools (Python)

For real-world LP problems, use industrial-grade solvers:

- `scipy.optimize.linprog` with the HiGHS backend — a state-of-the-art open-source LP/MIP solver.
- `cvxpy` — a modelling language for convex optimisation that supports LP, QP, and SOCP.
- `PuLP`, `OR-Tools` — higher-level modelling frameworks.
- `gurobipy` — commercial solver with a free academic licence.

Dual variables (shadow prices) are available via `result.eqlin.marginals` (for equality constraints) and `result.ineqlin.marginals`

Exercise 12.3 — Converting to Equality Form

Problem. Convert the LP: minimize $6x_1 + 5x_2$ subject to $3x_1 - 2x_2 \geq 5$, with x_1, x_2 free (unrestricted in sign), into equality form.

Solution.

1. **Flip the inequality**: multiply both sides by -1 to get $-3x_1 + 2x_2 \leq -5$.
2. **Split free variables**: write $x_i = x_i^+ - x_i^-$ with $x_i^+, x_i^- \geq 0$ for $i = 1, 2$.
3. **Add slack variable** $x_3 \geq 0$: $-3x_1^+ + 3x_1^- + 2x_2^+ - 2x_2^- + x_3 = -5$.
4. **Equality-form objective**: $6x_1^+ - 6x_1^- + 5x_2^+ - 5x_2^-$ (minimise, same value as before).

The equality-form LP has 5 non-negative variables $x_1^+, x_1^-, x_2^+, x_2^-, x_3 \geq 0$ and one equality constraint. Note that $x_3 = -3x_1^+ + 3x_1^- + 2x_2^+ - 2x_2^- + 5$; for this to be non-negative we need the original constraint $3x_1 - 2x_2 \geq 5$ to hold, which is exactly the feasibility condition.

Exercise 12.4 — One Simplex Pivot

Problem. Given $A \in \mathbb{R}^{4\times 6}$ with initial basis $\mathcal{B} = \{1, 2, 3, 4\}$: verify $A_{\mathcal{B}}$ is invertible, compute $x_{\mathcal{B}}$, $\boldsymbol{\mu}_{\mathcal{V}}$, and execute one pivot.

Solution sketch. Suppose we find $x_{\mathcal{B}} = [2, 3, 4, 5]^\top$ and $\boldsymbol{\mu}_{\mathcal{V}} = [-4/3, -5/3]^\top$ (both negative, so the vertex is suboptimal). Using Dantzig's rule, we choose $q = 6$ (the most negative reduced cost is $\mu_6 = -5/3$). The minimum ratio test (not shown here) yields $x'_6 = 3$ with leaving index $p = 4$. The new basis is $\mathcal{B}' = \{1, 2, 3, 6\}$.

Exercise 12.6 — Why the Minimum Ratio Test is Essential

Problem. LP with $A = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix}$, $b = [1, 2, 3]^\top$. Initial $\mathcal{B} = \{1, 2, 3\}$; introduce $q = 4$. What happens if we choose $p = 2$ instead of the correct leaving index $p = 1$?

Correct minimum ratio test. The direction vector is $d = A_{\mathcal{B}}^{-1} A_{\{4\}} = I\,[1, 1, 1]^\top = [1, 1, 1]^\top$. The ratios are $1/1 = 1$, $2/1 = 2$, $3/1 = 3$. The minimum is 1, achieved at $i = 1$, so the correct leaving index is $p = 1$.

What goes wrong if we choose $p = 2$ instead. The new basis would be $\mathcal{B}' = \{1, 3, 4\}$. Computing the new basic variable values: $x_{\mathcal{B}'} = A_{\{1,3,4\}}^{-1} b = [-1, 1, 2]^\top$. The value $x_1 = -1 < 0$ violates non-negativity! The resulting point is **infeasible**. The minimum ratio test exists precisely to prevent this. By always choosing the variable that hits zero first (the one with the smallest ratio), we guarantee that all basic variables remain non-negative after the pivot.

Exercise 12.7 — Log-Barrier Reformulation

Problem. Reformulate the LP $\text{minimize}_x\ c^\top x$ s.t. $Ax \geq 0$ as an unconstrained problem using a log-barrier penalty.

Solution. Replace each inequality constraint $(Ax)_i \geq 0$ with a penalty that goes to $+\infty$ as $(Ax)_i \to 0^+$:

$$\underset{x}{\text{minimize}}\ c^\top x - \mu \sum_{i=1}^{m} \log\big((Ax)_i\big)$$

where $\mu > 0$ (mu, "mew") is the *barrier parameter*. The log (logarithm) term is $-\infty$ if any $(Ax)_i \leq 0$, so the penalty forces the solution to stay strictly inside the feasible set. As $\mu \to 0^+$ (mu approaches zero from above), the penalty shrinks and the solution of the unconstrained problem approaches the LP optimum. This is the basis of *interior-point methods*: by gradually decreasing μ and solving the unconstrained problem at each value, we trace a path (the "central path") through the interior of the feasible set converging to the optimum.

Python: Full LP Workflow Demo

```python
import numpy as np
from scipy.optimize import linprog

# -- Example 12.9: Solve and extract dual certificate -------------------------
# LP:  min x^T c  s.t.  Ax = b,  x >= 0
A = np.array([[ 1.,  1., -1.],
              [-1.,  2.,  0.],
              [ 1.,  2.,  3.]])
b = np.array([1., -2., 5.])
c = np.array([1.,  1., -1.])

# Solve with scipy (standard form: Ax_eq = b, x >= 0)
res = linprog(c, A_eq=A, b_eq=b,
              bounds=[(0, None)]*3, method='highs')
print("Scipy solution:", res.x, "obj:", res.fun)

# -- Dual certificate ---------------------------------------------------------
lam = res.eqlin.marginals    # Lagrange multipliers for equality constraints
print("Dual lambda:", lam)
print("Dual obj b^T lam:", b @ lam)
print("Dual feasible A^T lam <= c:", np.all(A.T @ lam <= c + 1e-8))
print("Strong duality:", np.isclose(res.fun, b @ lam))

# -- Parametric LP: how does x* vary with b? ----------------------------------
for delta in [-1, 0, 1]:
    b_new = b + delta * np.array([1., 0., 0.])
    r = linprog(c, A_eq=A, b_eq=b_new,
```

Chapter 12 — Key Formulas Reference I

The following is a concise reference for all the major formulas developed in this chapter. Each formula is stated with its role in the simplex algorithm.

Problem Formulation

Equality form LP:

$$\underset{x}{\text{minimize}}\ c^\top x \ \text{ s.t. } \ Ax = b,\ x \geq 0.$$

The matrix $A \in \mathbb{R}^{m \times n}$ has m rows (constraints) and n columns (variables), with $m \leq n$.

Vertex and Basis Computations

Vertex extraction (basic variable values):

$$x_{\mathcal{B}} = A_{\mathcal{B}}^{-1} b.$$

Lagrange multipliers $\boldsymbol{\lambda}$ (lambda) at the current

Pivot Computations

Direction vector (how basics change as x_q enters):

$$d = A_{\mathcal{B}}^{-1} A_{\{q\}}.$$

Minimum ratio test (step size, leaving variable):

$$x'_q = \min_{i:\, d_i > 0} \frac{(x_{\mathcal{B}})_i}{d_i}.$$

Objective change per pivot:

$$c^\top x' - c^\top x = \mu_q\, x'_q.$$

Since $\mu_q < 0$ and $x'_q \geq 0$, each non-degenerate pivot decreases the objective.

Linear Programming: Six Essential Takeaways

1. An LP consists of a **linear objective** and **linear constraints**. Because both are linear, the feasible set is a convex polytope and the objective surface is flat, making the problem globally tractable.
2. Any LP in general form can be converted to **equality form** $Ax = b$, $x \geq 0$ using slack variables and variable splitting. This is the form required by simplex.
3. The feasible set is a **convex polytope**; the optimum always lies at a **vertex**. This is why the simplex algorithm — which examines vertices — is a complete and correct method.
4. The **simplex algorithm** maintains a basis partition $(\mathcal{B}, \mathcal{V})$ and moves between adjacent vertices by pivoting: the entering variable is chosen by Dantzig's or Bland's rule; the leaving variable is determined by the minimum ratio test.

Chapter 12 — Final Takeaways II

5. The **two-phase structure** handles initialisation: Phase 1 (auxiliary LP) finds a feasible starting vertex; Phase 2 (optimisation) pivots to the optimum. If Phase 1 fails ($z^* \neq 0$), the original LP is infeasible.

6. **Strong duality** ($p^* = d^*$) allows efficient optimality verification via **dual certificates**. The simplex algorithm delivers the optimal dual solution $\boldsymbol{\lambda}^*$ automatically. In Python, use `scipy.optimize.linprog` with the `'highs'` method and access dual variables via `res.eqlin.marginals`.

Next: Chapter 13 — Quadratic Programming